Intune-as-Code — Policy Management via Graph & Terraform (CI/CD)

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — Intune-as-Code (Policy Management via Graph & Terraform, CI/CD) 4
- What you get . . . 4
- What this delivers . . . 4
- The standard deployment model . . . 4
- Outcome . . . 4

1. Solution Overview — Intune-as-Code (Graph & Terraform, CI/CD) 5
- What Intune-as-code is . . . 5
- How it works . . . 5
- Two ways to talk to Graph . . . 5
- What you can control . . . 6
- Limitations (be honest with buyers) . . . 6
- Authentication and connections used . . . 6
- The standard deployment model . . . 6

2. Requirements — Intune-as-Code (Graph & Terraform, CI/CD) 7
- R1. Microsoft Intune licensed and in use . . . 7
- R2. Entra app registration with Graph application permissions . . . 7
- R3. Terraform Microsoft Graph provider available and pinned . . . 7
- R4. Microsoft.Graph PowerShell module . . . 8
- R5. Remote Terraform state backend with locking . . . 8
- R6. Development tenant or a designated pilot group . . . 8
- R7. Backup location and change-approval process . . . 8

Global Requirements — DevOps (category-level) 8
- G1. Target cloud subscription/tenant with sufficient rights . . . 9
- G2. IaC tooling installed and versioned . . . 9
- G3. Source control repository . . . 9
- G4. CI/CD platform and service connection . . . 9
- G5. Remote state / deployment backend . . . 9
- G6. Secrets management . . . 10
- G7. Naming, tagging, and region standards . . . 10
- G8. Change-control and non-production target . . . 10

3. Implementation — Intune-as-Code (Graph & Terraform, CI/CD) 10
- Naming you will create (reuse these exact names) . . . 10
- Step 1 — Assess: export the current tenant to code (read-only baseline) . . . 11
- Step 2 — Configure: register the automation identity . . . 11
- Step 3 — Configure: scaffold the Terraform repo . . . 11

Step 4 — Pilot: plan and apply to dev / pilot group (backup first) . . . 12
Step 5 — Validate: confirm the pilot . . . 12
Step 6 — Wire the CI/CD pipeline (validate → plan → approval → apply) . . . 12
Step 7 — Expand: promote to production, then widen assignments . . . 12
Step 8 — Operate: schedule drift detection and backup . . . 13
5. Testing — Intune-as-Code (Graph & Terraform, CI/CD) 13
Test 1 — Baseline export is complete . . . 13
Test 2 — Terraform initializes and validates . . . 13
Test 3 — Plan shows only intended changes (no surprise destroys) . . . 13
Test 4 — Policies exist and are pilot-scoped . . . 14
Test 5 — Policy applies on a pilot device . . . 14
Test 6 — Drift detection works . . . 14
Test 7 — Backup and restore round-trip . . . 14
Test 8 — Pipeline enforces plan and approval . . . 14
Exit criteria . . . 15
6. Troubleshooting — Intune-as-Code (Graph & Terraform, CI/CD) 15
Authentication / permissions . . . 15
Terraform init / provider . . . 15
Plan / apply . . . 15
Drift . . . 16
Backup / restore . . . 16
Assignments / devices . . . 16
Pipeline . . . 16
7. Best Practices — Intune-as-Code (Graph & Terraform, CI/CD) 17
Keep it clean . . . 17
Governance and decisions that belong to the customer . . . 17
Where to go next . . . 18
8. Deliverables Summary — Intune-as-Code (Graph & Terraform, CI/CD) 18
Documentation . . . 18
IaC and automation artifacts . . . 18
What the buyer receives . . . 19
Outcome . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary — Intune-as-Code (Policy Management via Graph & Terraform, CI/CD)

White-label · DevOps

A click-managed Intune tenant drifts. Policies get edited in the portal with no record of who changed what or why, there is no clean way to review a change before it hits every device, "test" and "production" tenants slowly diverge, and there is no reliable backup — so a bad edit or an accidental delete is hard to undo. This solution manages **Intune as code**: existing policies are exported to source, defined as **Terraform / Microsoft Graph** in Git, and deployed through a **CI/CD pipeline** with review, approval, backup, dev→prod promotion, and drift detection.

This is a complete, **deploy-it-today** package for putting Intune under source control and a pipeline, written for an administrator who knows the Intune portal but has never exported policies to code, used the Terraform Graph provider, or run a policy-as-code pipeline.

What you get

- A plain-language **Solution Overview** of the Intune-as-code model — baseline export, the policy-as-code repo, the CI/CD pipeline (validate → plan → review → deploy), dev→prod promotion, backup/restore, and drift detection.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (category-wide ones live in the separate Global Requirements).
- An **Implementation Guide** of precise, ordered steps — register the app, export the current tenant to code, structure the repo, plan against **dev**, review, then promote to **prod** behind approval.
- Five IaC / automation artifacts: a **PowerShell Graph export/discovery** script, the core **Terraform (**`main.tf`**)** for Intune policies, a **PowerShell validation** script, an `azure-pipelines.yml` pipeline (validate → plan → approval → apply), and a **PowerShell backup / drift report**.
- **Testing** and **Troubleshooting** guides for a confident, non-disruptive rollout.

What this delivers

Existing configuration, compliance, and app-protection policies **exported to code** as a versioned baseline; those policies defined and managed via the **Terraform Microsoft Graph provider** in Git; a **CI/CD pipeline** that runs `terraform plan` (the change preview), requires **review and approval**, then applies; a repeatable **dev→prod promotion** path; a **tenant backup/restore** workflow; and **drift detection** that flags portal edits made outside the pipeline — all proven against a **development tenant / test group** first.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — every change is plan-first (`terraform plan`), reviewed in a pull request, approval-gated, and piloted against a test group/dev tenant before production, with backup as the rollback.

Outcome

An Intune tenant managed like software — every policy exported, versioned, reviewed, and promoted through CI/CD with backup and drift detection — instead of untracked portal edits and quietly diverging tenants, deployable by a single administrator and validated before you scale.

1. Solution Overview — Intune-as-Code (Graph & Terraform, CI/CD)

What Intune-as-code is

Intune-as-code is the practice of defining your Microsoft Intune configuration — device configuration profiles, compliance policies, and app-protection/app-configuration policies — as **files in Git** rather than as clicks in the Intune admin center, and deploying them through a **CI/CD pipeline** instead of by hand. The configuration lives in **Terraform** using the **Microsoft Graph provider** (Terraform talks to the same Microsoft Graph API the portal uses), so every policy is versioned, reviewed in a pull request, previewed with `terraform plan`, and applied only after approval. The result is an Intune tenant you can review, promote from test to production cleanly, back up, and audit.

How it works

1. **Baseline export.** Before writing anything new, the current tenant is **exported to code** — existing configuration, compliance, and app policies are read from Graph and captured as the starting point, so you begin from reality, not a blank slate.

2. **Policy-as-code repo.** Policies are expressed as Terraform resources (the Microsoft Graph provider) and committed to Git. Terraform keeps a **state file** in a secure remote backend (per Global Requirements G5) recording what it manages.

3. **Plan is the preview.** `terraform plan` shows exactly what will be **created, changed, or destroyed** in Intune before anything is applied — the equivalent of a what-if.

4. **Review and approval.** Changes arrive as pull requests; the pipeline runs `validate` and `plan` and posts the plan for review; a human approves before `apply`.

5. **Dev→prod promotion.** The same code is applied first to a **development tenant (or a pilot group in production)**, validated, then promoted to production behind an approval gate — so test and prod stay in lock-step instead of drifting.

6. **Backup / restore.** A scheduled export captures the tenant's policies as JSON so a bad change or accidental delete can be restored.

7. **Drift detection.** A periodic `terraform plan` (with no code change) reveals any **portal edit made outside the pipeline** — drift — so it can be reconciled back into code or reverted.

Two ways to talk to Graph

• **Terraform Microsoft Graph provider** — the declarative core; policies as code with plan/apply and state. *Verify the current provider name and version — the Microsoft Graph / Intune Terraform provider ecosystem is evolving; confirm the provider and Intune resource support in your environment before relying on it.*

• **PowerShell + Microsoft Graph SDK** — used here for the **baseline export**, **backup**, and **drift reporting** (reading the tenant to JSON), which is often simpler than modeling every legacy policy in Terraform on day one. *Verify current `Microsoft.Graph` PowerShell module versions.*

What you can control

• **Which policies are code** — start with a scoped set (a few configuration + compliance + app-protection policies), then expand coverage.

• **Assignment targets** — pilot group first, then broader groups; assignments are part of the code.

• **Promotion path** — dev tenant / pilot group → production, gated by approval.

• **Backup cadence** — how often the tenant is exported to JSON.

• **Drift response** — reconcile portal changes into code, or revert them to match code.

Limitations (be honest with buyers)

• **Not every Intune setting is modeled** by the Terraform Graph provider yet, and some Graph endpoints are **beta**. Where a policy type is unsupported, manage it via the PowerShell/Graph export-and-apply path and note it. *Verify current provider resource coverage and whether an endpoint is beta before depending on it.*

• **App-protection and app-configuration policies** depend on app-registration and, for some scenarios, specific licensing — *verify current licensing and availability in your tenant.*

• `terraform destroy` and delete operations **remove real policies** from the tenant — this is why plan-first, review, approval, and backup are mandatory.

• A **development tenant** is ideal for promotion testing; where one is not available, pilot against a **test group** in production and expand assignments carefully.

• Managing Intune as code changes the operating model — admins must stop editing in the portal for managed policies, or drift detection will constantly flag their edits.

Authentication and connections used

• An **Entra app registration** (service principal) with the required **Microsoft Graph application permissions** (e.g. `DeviceManagementConfiguration.ReadWrite.All`, `DeviceManagementApps.ReadWrite.All`) granted admin consent — used by both Terraform and the PowerShell scripts. *Verify the current minimum permission set.*

• A **CI/CD service connection** using **workload identity federation (OIDC)** where possible, or the app registration's credential stored in the pipeline secret store / Key Vault (Global Requirements G4/G6) — never in code.

• A **remote Terraform state backend** (Azure Storage) with locking (Global Requirements G5).

The standard deployment model

Assess (export the current tenant to code as a baseline) → configure the repo, provider, and a scoped set of policies as Terraform → pilot by planning/applying to a dev tenant or a test group

with backup taken first → validate with the plan, the testing script, and a policy-assignment check → expand by promoting to production behind approval and widening assignments, with drift detection running on a schedule. *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, plan-first and approval-gated, with backup/restore as rollback.*

2. Requirements — Intune-as-Code (Graph & Terraform, CI/CD)

The category-wide prerequisites (target subscription/tenant with rights, IaC tooling, source control, CI/CD platform and service connection, remote state backend, secrets management, naming/tagging standards, and a non-production target) are covered once in **Global Requirements — DevOps** and are **not repeated here**. Confirm those first. Below are the prerequisites **specific to managing Intune as code**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Microsoft Intune licensed and in use

Description: An Intune-enabled tenant (Intune stand-alone, or via Microsoft 365 E3/E5 / EMS) with at least some enrolled devices or target groups. **Why required:** You are exporting and managing Intune policies; without Intune licensing and objects there is nothing to codify. **How to verify:** The **Intune admin center** (intune.microsoft.com) loads and shows **Devices** / **Configuration** / **Compliance**; users have Intune licenses. **How to implement if missing:** Assign Intune licenses and complete basic tenant setup before codifying policy. *Verify current licensing SKUs and availability in your tenant.*

R2. Entra app registration with Graph application permissions

Description: An Entra ID **app registration** (service principal) that Terraform and the PowerShell scripts authenticate as, with the Microsoft Graph **application** permissions to read and write Intune configuration, compliance, and app policies. **Why required:** Non-interactive pipeline runs and scripts need an app identity with least-privilege Graph rights; delegated/interactive auth does not work in CI/CD. **How to verify:** In **Entra ID** → **App registrations**, the app exists; under **API permissions** the required Graph application permissions (e.g. `DeviceManagementConfiguration.ReadWrite.All`, `DeviceManagementApps.ReadWrite.All`, `DeviceManagementManagedDevices.Read.All`) show **Granted** admin consent. *Verify the current minimum permission set.* **How to implement if missing:** Register the app, add the Graph application permissions, and have a Global/Privileged Role Administrator grant **admin consent**. Store its credential in Key Vault / the pipeline secret store, never in code.

R3. Terraform Microsoft Graph provider available and pinned

Description: The Terraform provider that manages Intune via Microsoft Graph, pinned to a specific version in the repo, plus the Terraform CLI (per Global Requirements G2). **Why required:** The provider is the declarative engine; Intune resource coverage and schema change between provider versions, so it must be pinned for reproducibility. **How to verify:** `terraform init` downloads the pinned provider and `terraform providers` lists it; the provider documents the Intune resources you need. *Verify the current provider name, version, and Intune resource coverage — the Microsoft Graph / Intune Terraform provider ecosystem is evolving and some endpoints are beta.* **How to implement if missing:** Add the

provider with a pinned version constraint in `versions.tf`, run `terraform init`, and confirm the resources you need are supported (fall back to the PowerShell/Graph path for unsupported types).

R4. Microsoft.Graph PowerShell module

Description: The `Microsoft.Graph` PowerShell SDK (used for baseline export, backup, and drift reporting). **Why required:** The export/backup/report scripts call Graph via the SDK to read the tenant to JSON. **How to verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a supported version; `Connect-MgGraph` succeeds with the app credential. *Verify current module versions — cmdlet surfaces change between versions.* **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser` and pin the version alongside the Terraform toolchain.

R5. Remote Terraform state backend with locking

Description: A secure remote backend for Terraform state (Azure Storage container) with locking and versioning (this is Global Requirements G5 applied to Intune-as-code — called out here because state is central to plan/apply/drift). **Why required:** State is how Terraform knows what it manages; local state is unsafe, unshareable, and makes drift detection unreliable. Losing state means Terraform tries to recreate existing policies. **How to verify:** The `backend "azurerm"` block resolves and `terraform init` connects to the container; the container has versioning enabled. **How to implement if missing:** Provision the storage account/container with encryption, versioning, and least-privilege access, and configure the backend before the first `apply`.

R6. Development tenant or a designated pilot group

Description: Either a **development Intune tenant** for promotion testing, or — if none exists — a designated **pilot Entra group** in production that changes are assigned to first. **Why required:** Policy changes affect real devices; dev→prod promotion and pilot-first rollout require a safe target before broad assignment. **How to verify:** A dev tenant is reachable with its own app registration, **or** a pilot group (e.g. `Intune-Pilot – Devices – Ring0`) exists and is used as the first assignment target. **How to implement if missing:** Obtain a dev tenant, or create the pilot group and make it the mandatory first assignment target in the code. *Which users/devices belong in the pilot ring is the customer's rollout decision.*

R7. Backup location and change-approval process

Description: A storage location for the tenant policy **backup** (JSON export) and a defined **approval** step (Azure DevOps Environment check or GitHub Environment required reviewers) for the production apply. **Why required:** Backup is the rollback for a bad change or accidental delete; approval is the control that keeps unreviewed changes out of production. **How to verify:** The backup script writes to the location; the `Prod` environment lists required reviewers and a test run pauses at the gate. **How to implement if missing:** Provision the backup location (secure, access-controlled) and configure the approval environment referenced by the pipeline's apply stage. *Who may approve production Intune changes is the customer's change-control decision.*

Global Requirements — DevOps (category-level)

These requirements apply to every solution in the DevOps category (Infrastructure-as-Code, pipelines, policy-as-code). They are published once at the category level so individual listings stay small. Confirm these before deploying any IaC/automation solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Infrastructure-as-code changes real cloud resources. Everything runs plan/what-if first, through a reviewed pipeline, against a non-production target before production — never an unreviewed apply against prod. State and pipeline credentials are treated as secrets.*

G1. Target cloud subscription/tenant with sufficient rights

Description: An Azure subscription (and Entra tenant) where resources will be created, with Owner or an equivalent scoped role at the deployment scope. Why: Landing-zone and policy deployments create management groups, role assignments, and resources that need elevated rights. Verify: `az account show` succeeds; the identity holds Owner/User Access Administrator at the target scope. Implement: Obtain a subscription and the required role assignment (or a service principal / workload identity with those rights).

G2. IaC tooling installed and versioned

Description: The required toolchain — Terraform CLI, Bicep CLI / Azure CLI, and Git — at pinned versions. Why: IaC is version-sensitive; provider/module/CLI drift breaks reproducibility. Verify: `terraform version`, `az bicep version`, `az version`, and `git --version` return supported builds. *Verify current provider/module versions.* Implement: Install and pin the toolchain versions; record them in the repo.

G3. Source control repository

Description: A Git repository (Azure Repos or GitHub) to hold the IaC and pipeline definitions. Why: Code review, history, and CI/CD all depend on version control; IaC not in Git defeats the purpose. Verify: The repo exists and the operator can clone/push. Implement: Create the repository and set branch protection before authoring infrastructure code.

G4. CI/CD platform and service connection

Description: A pipeline platform (Azure DevOps Pipelines or GitHub Actions) with a secure connection to the cloud (workload identity federation preferred over secrets). Why: Plan/apply must run through an auditable pipeline with least-privilege, not from a laptop. Verify: A pipeline can authenticate to the subscription via OIDC/workload identity or a scoped service principal. Implement: Configure the service connection using workload identity federation; scope its rights to the deployment target.

G5. Remote state / deployment backend

Description: A secure remote backend for state (e.g. Azure Storage for Terraform state) or Azure deployment stacks for Bicep. Why: Local state is unsafe and unshareable; remote state with locking prevents corruption and drift. Verify: The state storage account/container (or deployment-stack scope) exists and is access-controlled. Implement: Provision the state backend with encryption, versioning, and least-privilege access before first apply.

G6. Secrets management

Description: A secrets store (Azure Key Vault) for any credentials the pipeline needs; no secrets in code. Why: Committed secrets leak; IaC repos are frequently shared and reviewed. Verify: No secrets in the repo (scanning enabled); pipeline pulls secrets from Key Vault / secure variables. Implement: Move all secrets to Key Vault / pipeline secret store; enable secret scanning on the repo.

G7. Naming, tagging, and region standards

Description: An organizational naming convention, required tags, and allowed regions. Why: Consistent naming/tagging is needed for governance, cost management, and policy enforcement. Verify: A documented standard exists and is encoded in the IaC modules/policy. Implement: Define the standard and encode it as policy-as-code and module defaults.

G8. Change-control and non-production target

Description: A non-production subscription/scope to validate against, and a change-approval process for production. Why: Every change is validated with plan/what-if against non-prod and approved before prod apply. Verify: A non-prod target exists; the pipeline requires approval before prod stages. Implement: Provision a non-prod scope and add manual approval gates to the production pipeline stages. *Production change approval is the customer's operational decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Intune-as-Code (Graph & Terraform, CI/CD)

Follow these steps in order. Everything is **pilot-first**: you export the current tenant, model a scoped set of policies as code, plan and apply against a **dev tenant or a pilot group** with a backup taken first, validate, then promote to production behind approval and widen assignments. Every object you create is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later steps, testing, and the scripts. Placeholder tenant is `contoso.onmicrosoft.com`.

> *Run every change with `terraform plan` **first**. Never `apply` to production without an approved plan. Take a **backup** before any apply that changes production.*

Naming you will create (reuse these exact names)

Object	Name
Automation app registration (SPN)	`Intune-as-Code – Automation – SPN`
Pilot assignment group	`Intune-Pilot – Devices – Ring0`
Windows config profile (code)	`Windows Baseline – Corporate – Config`
Compliance policy (code)	`Compliance Minimum – Corporate – Compliance`

Object	Name
App-protection policy (code)	`App Protection – Mobile – MAM`
Terraform workspace — dev	`intune – Dev – Workspace`
Terraform workspace — prod	`intune – Prod – Workspace`
State container	`tfstate-intune`
Pipeline	`Intune-as-Code Pipeline – Endpoint – CICD`
Backup export folder	`backups/<yyyy-MM-dd>/`

Step 1 — Assess: export the current tenant to code (read-only baseline)

1. Sign in the automation identity for reading: `Connect-MgGraph` (or use the app credential from **R2**).
2. Run the export/discovery script from `scripts/`:

`powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Discovery.ps1 -OutputPath .\baseline` It reads existing **device configuration**, **compliance**, and **app-protection** policies to JSON under `.\baseline` and prints a summary count. This is your baseline and your first backup.

1. Review the exported JSON. Pick a **scoped starter set** to codify first — e.g. one Windows config profile, one compliance policy, one app-protection policy — rather than the whole tenant on day one.

Step 2 — Configure: register the automation identity

1. Confirm the app registration `Intune-as-Code – Automation – SPN` exists with the Graph application permissions and admin consent (**R2**).
2. Store its credential in Key Vault / the pipeline secret store (never in code). For local runs, set the standard `ARM_CLIENT_ID` / `ARM_TENANT_ID` (and the client secret or OIDC) environment variables the Terraform provider reads.
3. Confirm the `Microsoft.Graph` PowerShell module (**R4**) and Terraform CLI + pinned Graph provider (**R3**) are installed.

Step 3 — Configure: scaffold the Terraform repo

1. In the repo, add the provided files: `main.tf` (policy definitions), `versions.tf` (pinned provider + backend), and `azure-pipelines.yml`; put `IACINTUNE-Discovery.ps1`, `IACINTUNE-Validate.ps1`, and `IACINTUNE-Report.ps1` under `scripts/`.
2. Configure the **remote state backend** (`tfstate-intune` container, **R5**) in `versions.tf` and run `terraform init`.
3. Create two workspaces: `terraform workspace new "intune – Dev – Workspace"` and `terraform workspace new "intune – Prod – Workspace"` so dev and prod state never mix.
4. Translate your scoped starter set from the baseline JSON into Terraform resources in `main.tf`, using the names above (`Windows Baseline – Corporate – Config`, etc.). Assign them to `Intune-Pilot – Devices – Ring0` only.

5. Commit to a **feature branch** and open a pull request — Intune changes are reviewed, never pushed straight to `main`.

Step 4 — Pilot: plan and apply to dev / pilot group (backup first)

1. Take a backup: `powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -Backup -OutputPath .\backups`.

2. Select the dev workspace: `terraform workspace select "intune – Dev – Workspace"`.

3. **Plan (the preview):** `terraform plan -out tfplan`. Read it — it must show only the intended creates/changes for your starter set, assigned to the pilot group. **No unexpected destroys.**

4. Apply to dev / pilot: `terraform apply tfplan`.

5. In the Intune admin center (**Devices** → **Configuration** / **Compliance** / **Apps**) confirm the three named policies exist and are assigned to `Intune-Pilot – Devices – Ring0`.

Step 5 — Validate: confirm the pilot

1. Run the validation script:

`powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Validate.ps1 -PilotGroup "Intune-Pilot – Devices – Ring0"` It confirms the three policies exist, are assigned to the pilot group only, and that a fresh `terraform plan` shows **no drift** — see the **Testing** guide for pass/fail gates.

1. On a pilot device, confirm the config/compliance/app-protection policies apply as expected (sync from **Company Portal** / **Settings** → **Accounts** → **Access work or school**).

2. Resolve any issues and re-plan until clean before promoting.

Step 6 — Wire the CI/CD pipeline (validate → plan → approval → apply)

1. Create the pipeline `Intune-as-Code Pipeline – Endpoint – CICD` from `azure-pipelines.yml`.

2. Confirm the **service connection** uses workload identity federation (or the app credential from the secret store) and the `intune-prod` **environment** has the approval gate and reviewer group from **R7**.

3. The stages are: **Validate** (`terraform fmt`/`validate`) → **Plan** (posts the plan to the PR) → **Approval** (manual gate) → **Apply**. PRs run Validate + Plan; Apply only runs after approval.

Step 7 — Expand: promote to production, then widen assignments

1. Merge to `main`. The pipeline runs **Validate** → **Plan** against the **prod** workspace. Review the plan in the PR.

2. **Back up production first** (the pipeline's pre-apply step runs `IACINTUNE-Report.ps1 -Backup`), then approve the gate. The pipeline runs `terraform apply` against `intune – Prod – Workspace`.

3. Widen assignments in stages: after the pilot ring is healthy, add broader groups to the policy assignments **in code**, plan, review, and apply — never by editing the portal.

4. **Rollback** at any point: restore the affected policy from the most recent backup (`IACINTUNE-Report.ps1 -Restore`), or revert the code change and re-apply.

Step 8 — Operate: schedule drift detection and backup

1. Schedule a periodic pipeline run (or `IACINTUNE-Report.ps1 -DriftCheck`) that runs `terraform plan` with **no code change** — any non-empty plan is **drift** (a portal edit made outside the pipeline).

2. For each drift item, decide: **reconcile** (bring the portal change into code) or **revert** (re-apply code to overwrite it). Do not ignore drift — it is the signal that someone bypassed the pipeline.

3. Keep the scheduled **backup** running so a restore point always exists.

Onboard each new policy the same way: model it in `main.tf`, assign to the pilot ring, plan, review, apply, then widen. That repeatable loop is the payoff — Intune managed like software.

5. Testing — Intune-as-Code (Graph & Terraform, CI/CD)

Validate against a **dev tenant or the pilot group** before you promote to production. Each test has an explicit **pass/fail gate**. Reference the scripts by name; they are read-only / plan-only except the explicit backup/apply steps.

Test 1 — Baseline export is complete

• **How:** `powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Discovery.ps1 -OutputPath .\baseline`, then inspect `.\baseline`.

• **Pass:** JSON files exist for device configuration, compliance, and app-protection policies; the printed counts match what you see in the Intune admin center.

• **Fail:** Counts mismatch or empty → check the app's Graph permissions/admin consent (**R2**) and that `Connect-MgGraph` used the right identity.

Test 2 — Terraform initializes and validates

• **How:** `terraform init` then `terraform validate` and `terraform fmt -check`.

• **Pass:** `init` downloads the **pinned** Graph provider from `versions.tf`; `validate` returns success; formatting is clean.

• **Fail:** Provider not found → check the pinned provider name/version (**R3**). Backend error → check the `tfstate-intune` backend config (**R5**).

Test 3 — Plan shows only intended changes (no surprise destroys)

• **How:** `terraform workspace select "intune – Dev – Workspace"` then `terraform plan -out tfplan`.

• **Pass:** The plan shows only the creates/changes for your starter set (`Windows Baseline – Corporate – Config`, `Compliance Minimum – Corporate – Compliance`, `App Protection – Mobile – MAM`), assigned to the pilot group. **Zero unexpected** `destroy` **lines.**

• **Fail:** A `destroy` you did not intend → **stop**; usually a renamed resource or missing/incorrect state. Reconcile before applying.

Test 4 — Policies exist and are pilot-scoped

• **How:** `powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Validate.ps1 -PilotGroup "Intune-Pilot – Devices – Ring0"`, or the Intune admin center.

• **Pass:** The three named policies exist and are assigned **only** to `Intune-Pilot – Devices – Ring0` (not All Devices / All Users).

• **Fail:** Assigned to a broad group during pilot → fix the assignment in code, plan, and re-apply. Missing policy → re-run Step 4's apply.

Test 5 — Policy applies on a pilot device

• **How:** On an enrolled pilot device, sync (Company Portal / Access work or school) and check the config/compliance/app-protection result.

• **Pass:** The device receives the profiles; compliance evaluates; app-protection applies to the targeted app.

• **Fail:** Not applied → confirm the device is a member of the pilot group and licensed; allow time for Intune to deliver policy.

Test 6 — Drift detection works

• **How:** Make a small **portal** edit to one managed policy, then run `IACINTUNE-Report.ps1 -DriftCheck` (or `terraform plan`).

• **Pass:** The plan is **non-empty** and shows the portal edit as a pending change back to the coded value — drift is detected.

• **Fail:** Plan stays empty after a real portal change → state is stale or the resource is not actually managed; check the backend (**R5**) and that the policy is in `main.tf`.

Test 7 — Backup and restore round-trip

• **How:** `IACINTUNE-Report.ps1 -Backup -OutputPath .\backups`, then verify a `-Restore` dry run reads it back.

• **Pass:** A timestamped backup folder contains the policy JSON; the restore dry run reports it would recreate the exported policies.

• **Fail:** Backup empty or restore cannot parse → check the output path permissions and that the export completed (**R7**).

Test 8 — Pipeline enforces plan and approval

• **How:** Push a trivial change on a branch; open a PR; watch the pipeline.

• **Pass: Validate** and **Plan** run automatically and post the plan; the **Apply** stage **pauses at the approval gate** (**R7**) and the pre-apply **backup** runs before any apply.

• **Fail:** Apply runs without pausing → the environment approval check is not attached to the apply stage; fix the `intune-prod` environment configuration.

Exit criteria

All eight tests pass against dev / the pilot group, the plan is clean, drift is detected, backup/restore works, and the pipeline pauses for approval — **only then** promote to production and widen assignments.

6. Troubleshooting — Intune-as-Code (Graph & Terraform, CI/CD)

Common failures during an Intune-as-code rollout, as **symptom** → **cause** → **fix**. Re-run `IACINTUNE-Validate.ps1` after any fix to re-check the gates.

Authentication / permissions

Symptom: `Connect-MgGraph` or Terraform fails with `Authorization_RequestDenied` / `Insufficient privileges`. **Cause:** The automation app is missing a required Graph **application** permission or admin consent (**R2**). **Fix:** Add the required Graph application permissions (e.g. `DeviceManagementConfiguration.ReadWrite.All`, `DeviceManagementApps.ReadWrite.All`) and have an admin grant consent; re-run. *Verify the current minimum permission set.*

Symptom: Interactive auth works locally but the pipeline fails to authenticate. **Cause:** CI/CD needs a non-interactive identity; delegated auth does not work headless (**R2/G4**). **Fix:** Use workload identity federation (OIDC) or the app's client credential from the pipeline secret store — never a user login, never a secret in code.

Terraform init / provider

Symptom: `terraform init` cannot find the Microsoft Graph provider, or the resource types in `main.tf` are unknown. **Cause:** Wrong provider name/version, or the provider version does not model that Intune resource (**R3**). **Fix:** *Verify the current provider source, version, and Intune resource coverage;* pin the validated version in the `terraform{}` block. For unsupported policy types, manage them via the PowerShell/Graph path instead of forcing an unsupported resource.

Symptom: `Error acquiring the state lock` / backend errors on init. **Cause:** The remote state backend (`tfstate-intune`) is misconfigured or a previous run left a lock (**R5**). **Fix:** Confirm the storage account/container and access; if a stale lock remains from a crashed run, release it deliberately (`terraform force-unlock <id>`) only after confirming no run is active.

Plan / apply

Symptom: `terraform plan` shows unexpected **destroy** lines for policies you did not intend to remove. **Cause:** A resource was renamed (Terraform sees delete+create), or state is out of sync with the tenant. **Fix: Stop — do not apply.** For a rename, use `terraform state mv` / an import to reconcile; for stale state, import the existing policy. Re-plan until only intended changes remain.

Symptom: `apply` fails partway, leaving some policies created and others not. **Cause:** A mid-apply Graph error (throttling, a bad value, a beta endpoint change). **Fix:** Read the error, fix the offending resource, and re-`apply` — Terraform is idempotent and will converge. Check `/beta` endpoints for schema changes. *Verify whether the endpoint is beta.*

Symptom: Terraform reports "no changes" but the portal clearly differs. **Cause:** The differing policy is **not actually managed** by Terraform (not in `main.tf`), or state is stale. **Fix:** Add the policy to code and import it, or refresh state; unmanaged policies are invisible to plan by design.

Drift

Symptom: Drift detection constantly flags changes nobody deliberately made. **Cause:** Admins are still editing managed policies in the **portal**. **Fix:** Enforce the operating-model change — managed policies are edited only through Git/the pipeline. Reconcile the current drift (into code or revert) and communicate the new process.

Symptom: A legitimate portal change needs to stay. **Cause:** Someone made a valid change in the portal that should become the new baseline. **Fix: Reconcile** it into `main.tf` (update the code to match), plan, review, and apply — so code and tenant agree again and the plan goes empty.

Backup / restore

Symptom: `IACINTUNE-Report.ps1 -Backup` produces empty JSON files. **Cause:** Read permissions missing, or the endpoint is `/beta` and unavailable (**R7**). **Fix:** Confirm read scopes/consent and endpoint availability; *verify current beta endpoint paths.*

Symptom: You need to undo a bad production change fast. **Cause:** A change reached production and is causing device impact. **Fix:** Restore the affected policy from the most recent backup **or** revert the code change and re-apply through the pipeline (reviewed path). The pre-apply backup guarantees a restore point existed. Do not hand-edit in the portal — that creates drift.

Assignments / devices

Symptom: A new policy applied to far more devices than intended. **Cause:** It was assigned to All Devices/All Users instead of the pilot ring (Test 4 fail). **Fix:** Change the assignment in code to `Intune-Pilot – Devices – Ring0`, plan, and re-apply; widen assignments only after the pilot is healthy.

Symptom: A pilot device does not receive a policy. **Cause:** Device not in the pilot group, not licensed, or Intune has not yet delivered policy. **Fix:** Confirm group membership and licensing (**R1/R6**), force a sync, and allow time for delivery.

Pipeline

Symptom: The Apply stage runs without pausing for approval, or without a backup. **Cause:** The `intune-prod` environment approval check is missing, or the pre-apply backup step was removed (**R7**). **Fix:** Attach required reviewers to the `intune-prod` environment and confirm the deployment job runs the backup step before apply (Test 8).

7. Best Practices — Intune-as-Code (Graph & Terraform, CI/CD)

Keep it clean

• **Stop editing managed policies in the portal.** Once a policy is code, every change goes through Git and the pipeline. Portal edits become drift the pipeline will fight — the operating-model change is the whole point.

• **Plan is a gate, not a formality.** Read every `terraform plan` before approving. An unexpected **destroy** on a policy you did not intend to remove means **stop** and reconcile — usually a rename or stale state.

• **Backup before every production apply.** The pre-apply backup is your rollback. A restore point must always exist before Intune changes reach real devices.

• **Pilot ring first, always.** Assign new/changed policies to `Intune-Pilot – Devices – Ring0` first; widen assignments in code only after the ring is healthy. Never assign a new policy straight to All Devices/All Users.

• **Start scoped, expand coverage.** Don't try to codify the entire tenant on day one. Bring a few configuration, compliance, and app-protection policies under code, prove the loop, then add more.

• **Pin the toolchain.** Pin the Terraform CLI and the Graph provider version in `versions.tf`, and the `Microsoft.Graph` module version. Provider/schema drift silently changes behavior. *Verify current versions and Intune resource coverage periodically.*

• **Separate dev and prod state.** Use distinct workspaces (`intune – Dev – Workspace` / `intune – Prod – Workspace`) and a locked remote backend so environments never cross-contaminate.

• **Least-privilege app.** Grant the automation SPN only the Graph permissions it needs; store its credential in Key Vault / the pipeline secret store; prefer workload identity federation over a stored secret.

• **Mind unsupported settings and beta endpoints.** Where the Terraform Graph provider does not model a policy type, manage it via the PowerShell/Graph export-and-apply path and document it — don't force an unsupported resource. *Verify whether an endpoint is beta before depending on it.*

• **Run drift detection on a schedule.** A periodic no-change `terraform plan` surfaces anyone who bypassed the pipeline; act on drift (reconcile or revert) rather than letting it accumulate.

Governance and decisions that belong to the customer

• **Production change approval**, who may approve, and the promotion cadence are the customer's **change-control** decisions — the solution provides the technical gate.

• **Pilot-ring membership** (which users/devices are Ring0) is the customer's rollout decision.

• **Which settings each policy enforces** (security baseline strictness, compliance thresholds, app-protection rules) belongs to the customer's security/endpoint function; the code **implements** those choices.

Where to go next

Once Intune-as-code is live, adjacent catalog areas extend it:

- **DevOps** — the sibling *Azure Landing Zone with Bicep* / *…with Terraform* solutions to bring the surrounding Azure platform under the same code-and-pipeline discipline.
- **Intune / Endpoint** — configuration, compliance, and app-deployment solutions whose policies you can now manage as code instead of by hand.
- **Security / Identity** — Conditional Access and compliance-driven access solutions that consume the compliance signals these coded policies produce.
- **Backup / resilience** — extend the JSON backup into a scheduled, retained tenant-configuration backup with tested restore runbooks.

8. Deliverables Summary — Intune-as-Code (Graph & Terraform, CI/CD)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What Intune-as-code is — baseline export, policy-as-code repo, CI/CD (validate→plan→review→deploy), dev→prod promotion, backup/restore, drift detection; how Terraform Graph and plan work; benefits and limitations
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (DevOps)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered steps — export tenant, register SPN, scaffold repo, plan/apply to dev/pilot, wire pipeline, promote to prod, schedule drift/backup
5. Testing	Validation procedures with pass/fail gates
6. Troubleshooting	Field issues → cause → fix
7. Best Practices	Field-proven recommendations and where to go next

IaC and automation artifacts

File	What it does
`IACINTUNE-Discovery.ps1`	Read-only baseline **export** — reads existing device configuration, compliance, and app-protection policies from Microsoft Graph to JSON (your starting point and first backup)
`main.tf`	The core **Terraform** — defines the scoped starter set (`Windows Baseline – Corporate – Config`, `Compliance Minimum – Corporate – Compliance`, `App Protection – Mobile – MAM`) via the Microsoft Graph provider, assigned to the pilot group; includes `versions.tf`-style pinned provider + backend guidance
`IACINTUNE-Validate.ps1`	Read-only / plan validation — confirms the coded policies exist, are pilot-scoped, and that a fresh `terraform plan` shows no drift, against the pass/fail gates
`azure-pipelines.yml`	CI/CD pipeline (Azure DevOps; GitHub Actions notes inline) — Validate → Plan → **approval gate** → (pre-apply backup) → Apply, plan-first and approval-gated, with dev/prod workspaces
`IACINTUNE-Report.ps1`	**Backup / restore / drift** report — exports the tenant to a timestamped backup, checks for drift via `terraform plan`, and supports a restore dry run

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five IaC / automation artifacts (`main.tf`, `azure-pipelines.yml`, and three PowerShell scripts) in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — plan-first, approval-gated, with backup/restore as rollback.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

An Intune tenant managed like software — every policy exported, versioned, reviewed, and promoted through CI/CD with backup and drift detection — proven against a dev tenant / pilot group before it reaches production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IACINTUNE-Discovery.ps1

Purpose: Read-only baseline export for the "Intune-as-Code" solution. Reads existing device configuration profiles, compliance policies, and app-protection policies from Microsoft Graph and writes each to JSON under -OutputPath. This is your starting point before writing Terraform and doubles as your first backup. Makes NO changes to the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Discovery.ps1 -OutputPath .\baseline Requires the Microsoft.Graph PowerShell module. Authenticates as the automation app (Connect-MgGraph with the app credential) or interactively with the required scopes. Verify current Microsoft.Graph module versions and whether an endpoint is /beta.* EXPECTED RESULTS JSON files under .\baseline for configuration, compliance, and app-protection policies, plus a printed count per category. No tenant changes. .NOTES White-label. Read-only export. Makes no changes to Intune. > [CmdletBinding()] param([string]$OutputPath = ".\baseline", [string]$TenantId, [string]$ClientId, [string]$ClientSecret)

How to run: powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Discovery.ps1 -OutputPath .\baseline Requires the Microsoft.Graph PowerShell module. Authenticates as the automation app (Connect-MgGraph with the app credential) or interactively with the required scopes. Verify current Microsoft.Graph module versions and whether an endpoint is /beta.* EXPECTED RESULTS JSON files under .\baseline for configuration, compliance, and app-protection policies, plus a printed count per category. No tenant changes. .NOTES White-label. Read-only export. Makes no changes to Intune. > [CmdletBinding()] param([string]$OutputPath = ".\baseline", [string]$TenantId, [string]$ClientId, [string]$ClientSecret)

IACINTUNE-Report.ps1

Purpose: The rollback and drift toolkit for the "Intune-as-Code" solution: -Backup Export all managed policy types to a timestamped JSON backup (the rollback point). -Restore DRY-RUN by default — reports what a restore from a backup folder WOULD recreate. -DriftCheck Run `terraform plan` with no code change; a non-empty plan means drift (a portal edit made outside the pipeline). Backup and DriftCheck are read-only against the tenant. Restore is a safe preview unless you explicitly pass -Apply (gated). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -Backup -OutputPath .\backups powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -DriftCheck -TerraformDir .. powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -Restore -BackupFolder .\backups\2026-07-09 Requires the Microsoft.Graph module and (for -DriftCheck) the Terraform CLI. Verify current module / provider versions and whether an endpoint is /beta.* EXPECTED RESULTS A timestamped backup, a drift verdict, or a restore preview — no unintended changes. .NOTES White-label. Backup/DriftCheck are read-only. Restore defaults to a safe DRY-RUN; -Apply is gated. > [CmdletBinding(DefaultParameterSetName = 'Backup')] param([Parameter(ParameterSetName='Backup')] [switch]$Backup, [Parameter(ParameterSetName='Drift')] [switch]$DriftCheck, [Parameter(ParameterSetName='Restore')] [switch]$Restore, [string]$OutputPath = ".\backups", [string]$BackupFolder, [string]$TerraformDir = "..", [switch]$Apply # only meaningful with -Restore; gated, off by default)

How to run: powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -Backup -OutputPath .\backups powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -DriftCheck -TerraformDir ..

powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Report.ps1 -Restore -BackupFolder .\backups\2026-07-09 Requires the Microsoft.Graph module and (for -DriftCheck) the Terraform CLI. Verify current module / provider versions and whether an endpoint is /beta.* EXPECTED RESULTS A timestamped backup, a drift verdict, or a restore preview — no unintended changes. .NOTES White-label. Backup/DriftCheck are read-only. Restore defaults to a safe DRY-RUN; -Apply is gated. > [CmdletBinding(DefaultParameterSetName = 'Backup')] param([Parameter(ParameterSetName='Backup')] [switch]$Backup, [Parameter(ParameterSetName='Drift')] [switch]$DriftCheck, [Parameter(ParameterSetName='Restore')] [switch]$Restore, [string]$OutputPath = ".\backups", [string]$BackupFolder, [string]$TerraformDir = "..", [switch]$Apply # only meaningful with -Restore; gated, off by default)

IACINTUNE-Validate.ps1

Purpose: Confirms the Intune-as-code rollout matches the Testing guide's pass/fail gates: the three managed policies exist, are assigned ONLY to the pilot ring, Terraform validates, and a fresh `terraform plan` shows no drift. Read-only against Graph; plan-only against Terraform — makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Validate.ps1 -PilotGroup "Intune-Pilot – Devices – Ring0" Requires the Microsoft.Graph module and Terraform CLI. Authenticate to Graph first (Connect-MgGraph). *Verify current module / provider versions.* EXPECTED RESULTS A PASS/FAIL line per gate and the terraform plan summary. No changes are made. .NOTES White-label. Read-only / plan only. Does not apply. > [CmdletBinding()] param([string]$PilotGroup = "Intune-Pilot – Devices – Ring0", [string]$TerraformDir = "..", [string[]]$ExpectedPolicies = @("Windows Baseline – Corporate – Config", "Compliance Minimum – Corporate – Compliance", "App Protection – Mobile – MAM"))

How to run: powershell -ExecutionPolicy Bypass -File .\IACINTUNE-Validate.ps1 -PilotGroup "Intune-Pilot – Devices – Ring0" Requires the Microsoft.Graph module and Terraform CLI. Authenticate to Graph first (Connect-MgGraph). *Verify current module / provider versions.* EXPECTED RESULTS A PASS/FAIL line per gate and the terraform plan summary. No changes are made. .NOTES White-label. Read-only / plan only. Does not apply. > [CmdletBinding()] param([string]$PilotGroup = "Intune-Pilot – Devices – Ring0", [string]$TerraformDir = "..", [string[]]$ExpectedPolicies = @("Windows Baseline – Corporate – Config", "Compliance Minimum – Corporate – Compliance", "App Protection – Mobile – MAM"))

azure-pipelines.yml

Purpose: Plan-first, approval-gated delivery of the Intune Terraform. Stages: Validate -> terraform fmt + validate Plan -> terraform plan against the target workspace, published as the change preview on the PR Apply -> pre-apply BACKUP, then terraform apply, BEHIND a manual approval on the 'intune-prod' Environment (required reviewers = R7) HOW TO RUN Create the pipeline "Intune-as-Code Pipeline – Endpoint – CICD" from this file. Set the service connection (workload identity federation / app credential from the secret store — never in code) and add required reviewers to the 'intune-prod' Environment so Apply pauses for approval. EXPECTED RESULTS PRs run Validate + Plan automatically; the plan is attached. Apply only runs after approval, and always takes a backup first. Nothing applies unreviewed. White-label. Plan-first, approval-gated, backup-before-apply. GitHub Actions equivalent noted at the bottom. *Verify current task/provider/module versions.*
==========

How to run: Create the pipeline "Intune-as-Code Pipeline – Endpoint – CICD" from this file. Set the service connection (workload identity federation / app credential from the secret store — never in code) and add required reviewers to the 'intune-prod' Environment so Apply pauses for approval. EXPECTED

RESULTS PRs run Validate + Plan automatically; the plan is attached. Apply only runs after approval, and always takes a backup first. Nothing applies unreviewed. White-label. Plan-first, approval-gated, backup-before-apply. GitHub Actions equivalent noted at the bottom. *Verify current task/provider/module versions.*
==========

main.tf

Purpose: Defines a scoped starter set of Intune policies as code — one Windows device configuration profile, one compliance policy, and one app-protection policy — assigned to the pilot ring only. Managed through plan/apply with remote state. HOW TO RUN (plan FIRST, always) terraform init terraform workspace select "intune – Dev – Workspace" terraform plan -out tfplan # the change preview — read it before applying terraform apply tfplan # applies only what the plan showed EXPECTED RESULTS The three named policies created/updated in Intune and assigned to the pilot group. Re-running with no code change yields an empty plan (no drift). White-label. Plan/what-if first, through the reviewed pipeline. Back up before apply. *** VERIFY BEFORE USE** * The Microsoft Graph / Intune Terraform provider ecosystem is evolving and some Graph endpoints are /beta. Confirm the CURRENT provider name, version, resource names, and Intune resource coverage in your environment before relying on this. Where a policy type is unsupported by the provider, manage it via the PowerShell /Graph export-and-apply path (IACINTUNE-Discovery.ps1 / IACINTUNE-Report.ps1).